

CLOUD COMPUTING · AULA 8 · 8 DE OUTUBRO DE 2026

A loja na nuvem

A mesma Loja de Roupas que a turma programou em Coding, agora no ar: vitrine no S3, regras numa função Lambda e uma API de verdade na frente, com o API Gateway.

Três regras para ninguém ficar para trás

REGRA 1

Faça junto, no seu console

Cada fase é feita na sua conta AWS, enquanto o professor faz na dele, na tela.

REGRA 2

Fase só acaba no Confira

Cada fase termina com algo que dá para ver funcionando: uma página, uma resposta, um log.

REGRA 3

Travou? Arquivo pronto

Os arquivos de cada fase estão em `loja-na-nuvem-arquivos.zip`, no site. Copie, cole e volte para a turma.

ATALHO

Aperte `M` a qualquer momento para ver o mapa e onde estamos.



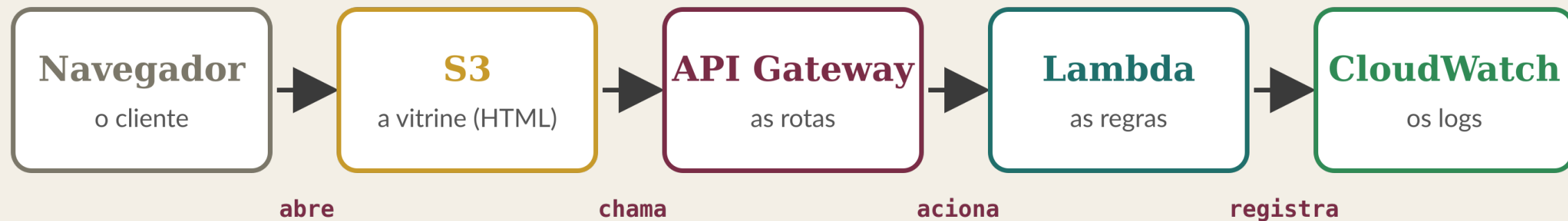
O MAPA DE HOJE

Nove fases, três mundos

O primeiro mundo revisa a base, o segundo revisa a Lambda e o terceiro é o conteúdo novo: a API. Os losangos são os chefes de cada mundo.

O QUE VAMOS CONSTRUIR

A loja, no fim da aula



- ◆ O **S3** entrega a página da loja. Arquivo parado, sem servidor ligado.
- ◆ O **API Gateway** recebe `GET /produtos` e `POST /carrinho/total` e entrega cada pedido para a **Lambda**.
- ◆ A **Lambda** roda o mesmo cálculo de frete que a turma testou em Coding: grátis a partir de R\$ 200.
- ◆ Nenhum servidor fica ligado esperando cliente. Paga-se pelo que é usado.



FASE 1-1 · REVISITA AULAS 1 A 4

Conta, região e trava de custo

Antes de criar qualquer recurso: a conta protegida, a região certa e um alarme para o cartão não levar susto.

AO FINAL DA FASE

console na região N. Virginia e alerta de orçamento conferido

O que é nuvem

Usar computação de outra empresa **pela rede**, sob demanda, pagando pelo que usar. O NIST resume em cinco características.

PENSE ASSIM

Energia elétrica: ninguém tem usina em casa; liga na tomada e paga a conta do que gastou.

CUIDADO

Servidor alugado por um ano, sem elasticidade e sem medição, não é nuvem: é hospedagem.

exemplo

- | | |
|--------------------------------|-----------------------------------|
| 1. Autoatendimento sob demanda | (você cria sozinho, na hora) |
| 2. Acesso amplo pela rede | (de qualquer lugar) |
| 3. Recursos compartilhados | (vários clientes, mesmo hardware) |
| 4. Elasticidade rápida | (cresce e encolhe) |
| 5. Serviço medido | (paga pelo uso) |

Camadas de serviço

Quanto mais alta a camada, menos coisa fica sob sua responsabilidade e mais a AWS cuida por você.

CUIDADO

Responsabilidade compartilhada: a AWS protege a nuvem; você protege o que coloca nela, como um bucket público por engano.



exemplo

IaaS	EC2	you take care of the operating system and everything above it
PaaS	Elastic Beanstalk	you take care of the application
FaaS	Lambda	you take care of only the function
SaaS	Gmail	you just use it

Today: S3 (storage), Lambda (FaaS) and API Gateway (managed service)

Região e zona de disponibilidade

Região é um lugar do mundo com vários data centers da AWS. **Zona de disponibilidade** é um desses data centers, isolado dos outros. Cada recurso nasce numa região.

CAUIDADO

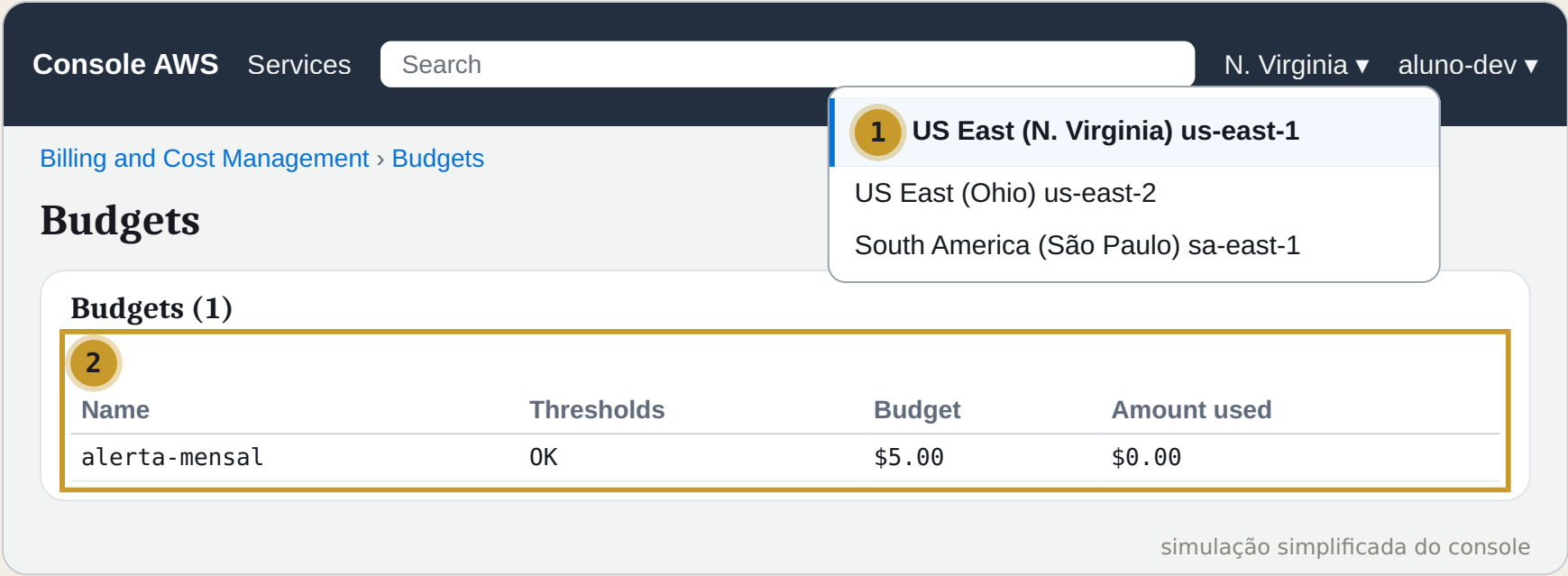
"Sumiu minha função!" quase sempre é região errada no canto superior direito do console.

exemplo

us-east-1	N. Virginia (EUA)	mais serviços, preços menores
sa-east-1	São Paulo	menor latência no Brasil, mais caro

Hoje, todos em us-east-1: o recurso que você criar só aparece na região onde foi criado.

Conferir antes de criar



- 1 No canto superior direito, escolha **N. Virginia (us-east-1)**.
- 2 Em **Billing and Cost Management > Budgets**, confira o alerta de orçamento da Atividade 1.
- 3 Entre sempre com o **usuário do dia a dia**, nunca com o usuário raiz.

Sem conta ainda? Forme dupla com um colega. Contas criadas a partir de 15/07/2025 entram num plano com créditos; a Lambda segue com 1 milhão de chamadas por mês sempre grátis.

Checkpoint: base pronta

- ◆ Região N. Virginia no canto do console.
- ◆ Alerta de orçamento existindo em Budgets.
- ◆ Usuário do dia a dia, com segundo fator.

CONFIRA

O nome da região aparece como **N. Virginia** em todas as telas do console.

TRAVOU NESTA FASE?

Siga a **Atividade 1: conta AWS com trava de orçamento**, no site. Enquanto isso, acompanhe a aula com um colega.



FASE 1-2 · REVISITA AULA 6

A vitrine no S3

O S3 guarda arquivos e também entrega um site estático, sem servidor ligado.

AO FINAL DA FASE

a página da loja abrindo pelo endereço do S3

Bucket e objeto

Bucket é um contêiner de arquivos com nome único no mundo. Cada arquivo dentro dele é um **objeto**, com uma chave (o nome) e o conteúdo.

PENSE ASSIM

Uma gaveta com etiqueta única no mundo inteiro; cada papel dentro dela é um objeto.

CUIDADO

Nome de bucket: só minúsculas, números e hífen, e ninguém no mundo pode ter usado antes.

exemplo

```
bucket:  loja-roupas-ana-2026
objeto:  index.html
endereço do site:
http://loja-roupas-ana-2026.s3-website-us-east-1.amazonaws.com
```

index.html: a vitrine, por enquanto fixa

```
index.html

<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <title>Loja de Roupas</title>
  <style>
    body { font-family: sans-serif; max-width: 720px; margin: 40px auto; }
    .peca { border: 1px solid #ccc; border-radius: 8px; padding: 12px; margin: 8px 0; }
  </style>
</head>
<body>
  <h1>Loja de Roupas</h1>
  <p>Vitrine estática, servida pelo Amazon S3.</p>
  <div class="peca">Camiseta básica (M): R$ 39,90</div>
  <div class="peca">Calça jeans (G): R$ 129,90</div>
  <div class="peca">Moletom (P): R$ 159,90</div>
</body>
</html>
```

- ◆ Crie o arquivo no VS Code. Os preços estão escritos à mão; na Fase 3-2 eles virão da API.
- ◆ Abra o arquivo no navegador para conferir antes de subir.

Criar o bucket

Console AWS

Services

Search

N. Virginia ▼

aluno-dev ▼

[Amazon S3](#) › [Buckets](#) › [Create bucket](#)

Create bucket

General configuration

General purpose

Directory

1

Bucket name

Só minúsculas, números e hífen. Único no mundo.

loja-roupas-ana-2026

Block Public Access settings for this bucket

2

☐ Block all public access

Turning off block all public access might result in this bucket and the objects within becoming public.

3

☒ I acknowledge that the current settings might result in this bucket and the objects within becoming public.

Cancel

4

Create bucket

- 1 Nome `loja-roupas-SEUNOME-2026`. A região vem do canto superior: **N. Virginia**.
- 2 Desmarque **Block all public access**.
- 3 Marque o aviso de que o bucket pode ficar público.
- 4 Clique em **Create bucket**, no fim da página.

Depois: entre no bucket › **Upload** › **Add files** › `index.html` › **Upload**.

Ligar o site estático

Console AWS Services Search N. Virginia aluno-dev

Amazon S3 > Buckets > loja-roupas-ana-2026 > Edit static website hosting

Edit static website hosting

Static website hosting

1 Disable **Enable** Host a static website Redirect requests

2 Index document

index.html

Cancel 3 Save changes

Depois, em Properties > Bucket website endpoint

4 <http://loja-roupas-ana-2026.s3-website-us-east-1.amazonaws.com>

simulação simplificada do console

- 1 Aba Properties > Static website hosting > Edit > Enable.
- 2 Documento de índice: `index.html`.
- 3 Save changes.
- 4 O endereço do site aparece no fim de Properties. Ainda dá 403: falta a política.

A política que libera a leitura

```
bucket policy

{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "LeituraPublica",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::NOME-DO-BUCKET/*"
  }]
}
```

- ◆ `Principal: "*" : qualquer pessoa.`
- ◆ `s3:GetObject` : só ler arquivos, nunca apagar ou enviar.
- ◆ Troque `NOME-DO-BUCKET` pelo seu. O `/*` no fim vale para todos os arquivos do bucket.

Colar a política

Console AWS

Services

Search

N. Virginia ▼

aluno-dev ▼

[Amazon S3](#) > [Buckets](#) > [loja-roupas-ana-2026](#) > [Edit bucket policy](#)

Edit bucket policy

Bucket policy

Bucket ARN

arn:aws:s3:::loja-roupas-ana-2026

1

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "LeituraPublica",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::loja-roupas-ana-2026/*"
  }]
}
```

Cancel

2

Save changes

simulação simplificada do console

- 1 Aba **Permissions** > **Bucket policy** > **Edit**. Cole a política com o **seu** nome de bucket.
- 2 **Save changes**. Recarregue o endereço do site: a vitrine aparece.

Erro "Access denied" ao salvar? O **Block all public access** ainda está marcado.

O endereço do site mostra 403 Forbidden. Por quê?

```
terminal  
  
$ curl -I http://loja-roupas-ana-2026.s3-website-us-east-1.amazonaws.com  
HTTP/1.1 403 Forbidden
```

Quase sempre é uma destas duas: o **Block all public access** continua marcado, ou a **política** não foi salva (ou tem o nome do bucket errado).

403 quer dizer "existe, mas você não tem permissão". 404 seria "não existe": aí o problema é o nome do arquivo.

FASE 1-2 • CONCLUÍDA

Checkpoint: vitrine no ar

endereço

`http://NOME-DO-BUCKET.s3-website-us-east-1.amazonaws.com`

CONFIRA

O endereço do site abre a página **Loja de Roupas** com as três peças.

TRAVOU NESTA FASE?

Baixe `loja-na-nuvem-arquivos.zip` em Materiais › Cloud, no site, e use o arquivo da pasta desta fase. Pasta `fase-1-2`: `index.html` e `politica-bucket.json`. O passo a passo detalhado está no **Roteiro completo**, passos 11 e 12.



FASE 1-3 · REVISITA AULAS 3 E 6

Quanto custa a vitrine

Na nuvem, cada decisão técnica tem um preço. Antes de seguir, a conta da vitrine.

AO FINAL DA FASE
a conta de quanto a vitrine custa por mês

Pagar pelo uso

Na nuvem, o custo vem do uso: quanto se guarda, quantas vezes se acessa e quanto dado sai para a internet. Gasto de capital (comprar servidor) vira gasto operacional (conta mensal).

CUIDADO

Barato não é grátis: um bucket esquecido com muitos arquivos vira cobrança todo mês.

exemplo

CAPEX comprar servidor: paga tudo antes, usando ou não
OPEX nuvem: paga por mês, pelo que usou

S3 Standard (us-east-1), valores de referência:

armazenamento	US\$ 0,023 por GB por mês
leitura (GET)	US\$ 0,0004 por 1.000 pedidos

A vitrine com 10 mil visitas por mês

ITEM	CONTA	CUSTO POR MÊS
Armazenamento	1 arquivo de 2 KB, praticamente 0 GB	US\$ 0,00
Leituras	10.000 visitas × 1 GET = 10.000 pedidos → 10 × US\$ 0,0004	US\$ 0,004
Transferência	10.000 × 2 KB = 20 MB, dentro da faixa gratuita mensal	US\$ 0,00
Total		menos de 1 centavo de dólar

PARA O RELATÓRIO

Anote a conta no relatório de custo da equipe. Com fotos dos produtos, o armazenamento e a transferência crescem: refaça a conta com o tamanho real.

Checkpoint: custo anotado

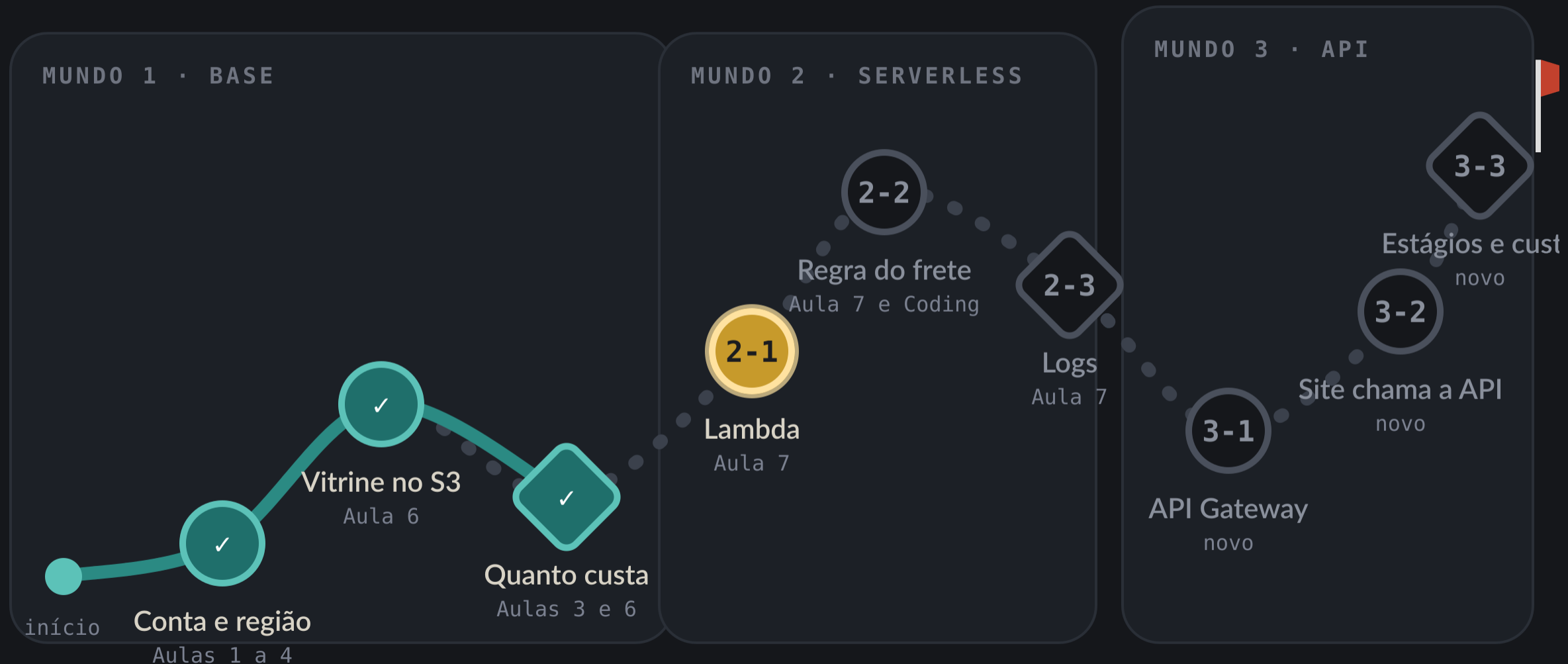
- ◆ A conta da vitrine no relatório de custo.
- ◆ A suposição de visitas escrita ao lado do número.

CONFIRA

O relatório mostra a conta com o número de visitas usado.

TRAVOU NESTA FASE?

Use a tabela do slide anterior como modelo e troque só o número de visitas.



FASE 2-1 · REVISITA AULA 7

A primeira Lambda da loja

A lista de produtos sai do HTML e passa a vir de uma função na nuvem.

AO FINAL DA FASE
a função loja-api devolvendo os produtos em JSON

Função como serviço (Lambda)

Você escreve só a função; a AWS cuida do servidor, liga quando chega um **evento** e desliga depois. Cobra por chamada e pelo tempo de execução.

PENSE ASSIM

Um caixa que só aparece quando chega cliente, atende e vai embora. Você paga pelos minutos de atendimento.

CUIDADO

exemplo

```
evento chega -> Lambda acorda -> roda lambda_handler(event, context)  
                                     -> devolve a resposta -> dorme
```

Sempre grátis: 1 milhão de chamadas por mês

A Lambda não guarda nada entre uma chamada e outra. Dado que precisa durar vai para um banco ou para o S3.

JSON

Formato de texto para trocar dados entre programas. Parece um dicionário do Python: chaves entre aspas, valores, listas entre colchetes.

CUIDADO

JSON usa aspas duplas e `true` / `false` minúsculos. Aspas simples quebram o formato.

exemplo

```
[  
  {"id": 1, "nome": "Camiseta básica", "preco": 39.9},  
  {"id": 2, "nome": "Calça jeans", "preco": 129.9}  
]
```

```
json.dumps(dados)    # Python -> texto JSON  
json.loads(texto)    # texto JSON -> Python
```

Criar a função loja-api

Console AWS Services Search N. Virginia aluno-dev

Lambda > Functions > Create function

Create function

1 Author from scratch
Começar com um exemplo simples.

Use a blueprint
Partir de um modelo pronto.

Container image
Usar uma imagem de contêiner.

Basic information

2 Function name
loja-api

3 Runtime
Python 3.13

Architecture
x86_64

Execution role
Create a new role with basic Lambda permissions

Cancel

4 Create function

simulação simplificada do console

- 1 Author from scratch.
- 2 Nome: loja-api.
- 3 Runtime Python 3.13 (ou o Python mais novo da lista).
- 4 Deixe a permissão padrão e clique em Create function.

Primeira versão: só a vitrine

```
lambda_function.py

import json

PRODUTOS = [
    {"id": 1, "nome": "Camiseta básica", "preco": 39.90, "tamanho": "M"},
    {"id": 2, "nome": "Calça jeans", "preco": 129.90, "tamanho": "G"},
    {"id": 3, "nome": "Moletom", "preco": 159.90, "tamanho": "P"},
]

def resposta(status, dados):
    return {
        "statusCode": status,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps(dados, ensure_ascii=False),
    }

def lambda_handler(event, context):
    return resposta(200, PRODUTOS)
```

- ◆ **PRODUTOS** é a mesma vitrine do Coding, agora numa lista de dicionários.
- ◆ **resposta** monta o formato que a AWS espera: código HTTP, cabeçalhos e o corpo em texto JSON.
- ◆ Sem **Deploy**, a função continua rodando o código antigo.

Colar o código e publicar

Console AWS Services Search N. Virginia aluno-dev

Lambda > Functions > loja-api

loja-api

Code Test Monitor Configuration

Code source · lambda_function.py

2 Deploy Test

1

```
import json

PRODUTOS = [
    {"id": 1, "nome": "Camiseta básica", "preco": 39.90, "tamanho": "M"},
    {"id": 2, "nome": "Calça jeans", "preco": 129.90, "tamanho": "G"},
    {"id": 3, "nome": "Moletom", "preco": 159.90, "tamanho": "P"},
]

...
```

Successfully updated the function loja-api.

simulação simplificada do console

- 1 Em `lambda_function.py`, apague tudo e cole o código do slide anterior.
- 2 Clique em **Deploy** e espere a faixa verde.

Sem **Deploy**, o teste roda o código antigo, o "Hello from Lambda!".

Evento de teste

```
evento de teste

{
  "routeKey": "GET /produtos"
}
```

- ◆ Aba **Test** › **Create new event**, nome `produtos`, cole o JSON e salve. Clique em **Test**.
- ◆ Esse é o formato que o API Gateway vai mandar na Fase 3-1. Por enquanto, a função ainda nem olha para ele.
- ◆ Resultado esperado: `statusCode: 200` e o `body` com as três peças.

Rodar o teste

Console AWS Services Search N. Virginia aluno-dev

Lambda > Functions > loja-api

loja-api

Code **Test** Monitor Configuration

Test event

Create new event Edit saved event

1 Event name

produtos

```
{
  "routeKey": "GET /produtos"
}
```

2

3 Save Test

Executing function: succeeded

4

```
{"statusCode": 200, "body": "[{\"id\": 1, \"nome\": \"Camiseta básica\", ...}]"}
```

- 1 Aba **Test**, evento novo com nome `produtos`.
- 2 Cole o JSON do evento.
- 3 **Save** e depois **Test**.
- 4 Abra **Details**: `statusCode` 200 e as três peças no `body`.

Checkpoint: função respondendo

● ● ● resultado do teste

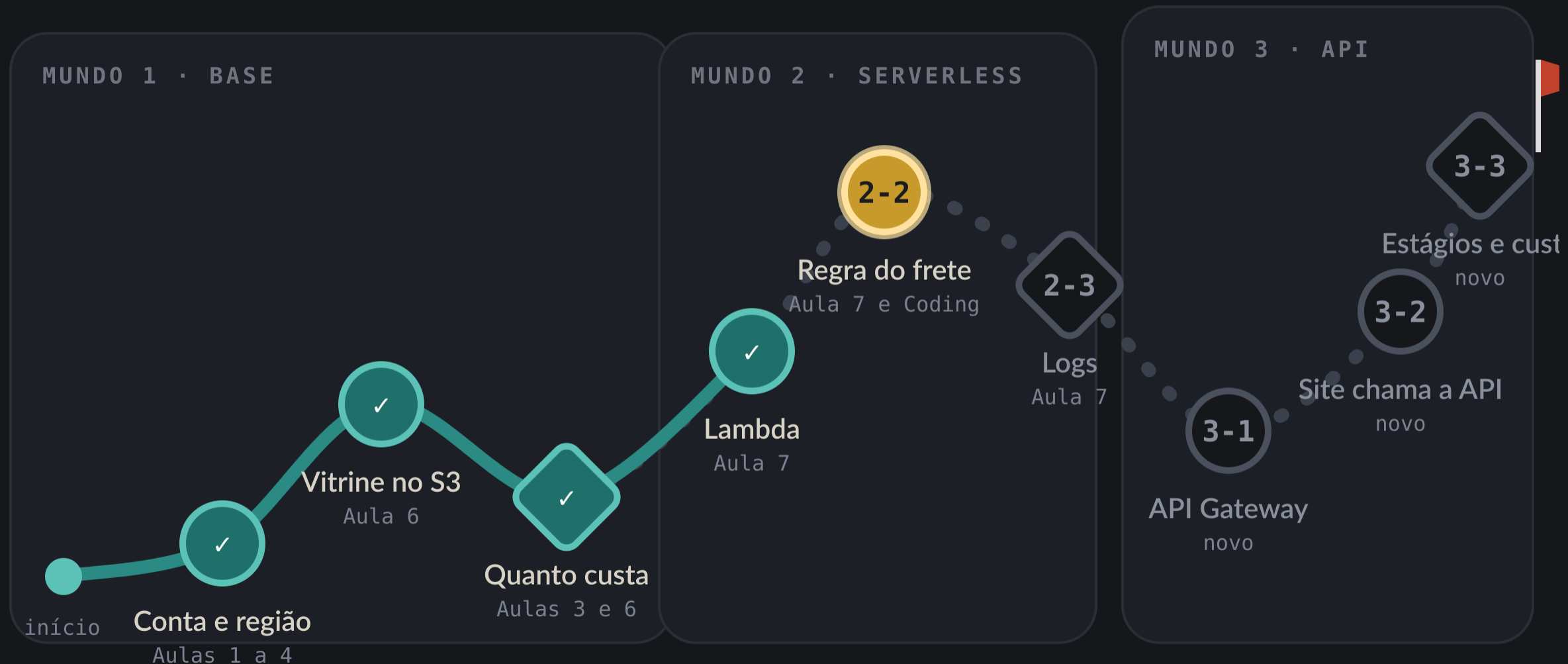
```
{
  "statusCode": 200,
  "headers": {"Content-Type": "application/json"},
  "body": "[{\"id\": 1, \"nome\": \"Camiseta básica\", ...}"
}
```

CONFIRA

O teste no console mostra **Succeeded** e o corpo com as três peças.

TRAVOU NESTA FASE?

Baixe `loja-na-nuvem-arquivos.zip` em Materiais › Cloud, no site, e use o arquivo da pasta desta fase. Pasta `fase-2-1`: `lambda_function.py` e `evento-produtos.json`.



FASE 2-2 · REVISITA AULA 7 E CODING

A regra do frete na nuvem

A mesma regra que a turma testou em Coding, agora atendendo pedidos pela rede.

AO FINAL DA FASE

a função calculando o total do carrinho com frete

A mesma regra, outro lugar

CODING · loja/calculos.py

Testada com pytest

```
loja/calculos.py

def frete(valor_da_compra):
    if valor_da_compra >= FRETE_GRATIS_A_PARTIR_DE:
        return 0.0
    return FRETE_FIXO
```

CLOUD · lambda_function.py

Respondendo pela rede

```
lambda_function.py

def frete(valor):
    if valor >= FRETE_GRATIS_A_PARTIR_DE:
        return 0.0
    return FRETE_FIXO
```

Regra de negócio não muda de lugar para lugar. O que muda é **quem chama**: um teste, um terminal ou um cliente pela internet.

O arquivo novo, parte 1: dados e regra

lambda_function.py (parte 1 de 2)

```
import json

PRODUTOS = [
    {"id": 1, "nome": "Camiseta básica", "preco": 39.90, "tamanho": "M"},
    {"id": 2, "nome": "Calça jeans", "preco": 129.90, "tamanho": "G"},
    {"id": 3, "nome": "Moletom", "preco": 159.90, "tamanho": "P"},
]

FRETE_FIXO = 15.0
FRETE_GRATIS_A_PARTIR_DE = 200.0

def frete(valor):
    if valor >= FRETE_GRATIS_A_PARTIR_DE:
        return 0.0
    return FRETE_FIXO

def resposta(status, dados):
    return {
        "statusCode": status,
        "headers": {"Content-Type": "application/json"},
        "body": json.dumps(dados, ensure_ascii=False),
    }
```

- ◆ Troque **todo** o código da função: comece por esta parte.
- ◆ `frete` é a mesma regra de `loja/calculos.py`, do Coding.
- ◆ `resposta` continua igual à da Fase 2-1.

Parte 2: duas rotas numa função

```
● ● ● lambda_function.py (parte 2 de 2)

def lambda_handler(event, context):
    rota = event.get("routeKey")          # ex.: "GET /produtos"
    print("pedido:", rota)                # aparece no CloudWatch

    if rota == "GET /produtos":
        return resposta(200, PRODUTOS)

    if rota == "POST /carrinho/total":
        itens = json.loads(event.get("body") or "[]")
        precos = {p["id"]: p["preco"] for p in PRODUTOS}
        subtotal = sum(precos[i["id"]] * i["quantidade"] for i in itens)
        return resposta(200, {
            "subtotal": round(subtotal, 2),
            "frete": frete(subtotal),
            "total": round(subtotal + frete(subtotal), 2),
        })

    return resposta(404, {"erro": "rota não encontrada"})
```

- ◆ `routeKey` é o "método + caminho" que o API Gateway manda: `GET /produtos` ou `POST /carrinho/total`.
- ◆ O corpo do POST chega como **texto**: `json.loads` transforma em lista de itens.
- ◆ O `print` vai para os logs do CloudWatch (Fase 2-3). Qualquer outra rota devolve 404.
- ◆ Cole embaixo da parte 1 e clique em **Deploy**.

Evento de teste do carrinho

evento de teste

```
{
  "routeKey": "POST /carrinho/total",
  "body": "[{\"id\": 1, \"quantidade\": 3}, {\"id\": 2, \"quantidade\": 1}]"
}
```

- ◆ Crie um segundo evento, `carrinho`, com este JSON, e teste.
- ◆ Repare: o `body` é um **texto** com JSON dentro, por isso as aspas aparecem com barra.
- ◆ Esperado: `{"subtotal": 249.6, "frete": 0.0, "total": 249.6}`.

E com uma camiseta só?

body do evento

```
"[{\"id\": 1, \"quantidade\": 1}]"
```

```
{"subtotal": 39.9, "frete": 15.0,  
"total": 54.9}
```

Abaixo de R\$ 200, o frete de R\$ 15 entra. É o mesmo resultado do teste

`test_abaixo_de_200_paga_frete` de Coding.

FASE 2-2 • CONCLUÍDA

Checkpoint: carrinho calculado

resultado

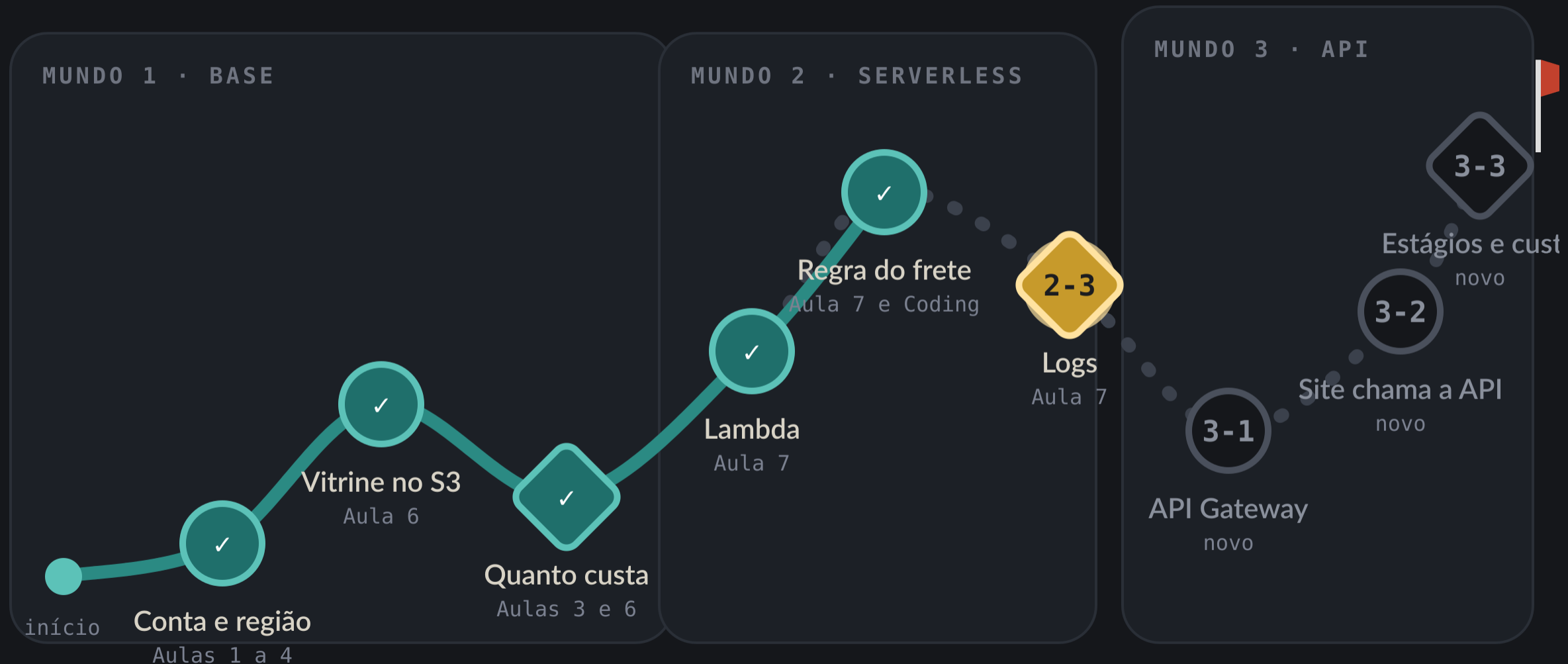
```
{"subtotal": 249.6, "frete": 0.0, "total": 249.6}
```

CONFIRA

Os dois eventos, `produtos` e `carrinho`, passam no teste do console.

TRAVOU NESTA FASE?

Baixe `loja-na-nuvem-arquivos.zip` em Materiais › Cloud, no site, e use o arquivo da pasta desta fase. Pasta `fase-2-2`: `lambda_function.py` e `evento-carrinho.json`.



FASE 2-3 · REVISITA AULA 7

Logs e custo da função

O que a função faz quando ninguém está olhando fica registrado nos logs.

AO FINAL DA FASE

o print da função aparecendo no CloudWatch

Início a frio

Na primeira chamada depois de um tempo parada, a AWS precisa preparar o ambiente da função. Essa chamada demora mais. As seguintes reaproveitam o ambiente.

CUIDADO

Para a maioria dos sites, 100 ms a mais na primeira chamada é imperceptível. Para um jogo em tempo real, pode não ser.

exemplo

```
REPORT Duration: 2.1 ms  Billed Duration: 3 ms  Memory: 128 MB  
      Init Duration: 98.4 ms  <- só na primeira chamada
```

Ler os logs

Console AWS

Services

Search

N. Virginia

aluno-dev

CloudWatch

Log groups

/aws/lambda/loja-api

loja-api · aba Monitor

1

View CloudWatch logs

Log streams (2)

2

Log stream	Last event time
2026/10/08/[\$LATEST]7f3c91e2	2026-10-08 19:42:10
2026/10/08/[\$LATEST]1a0b44d8	2026-10-08 19:31:55

Log events

3

Message
START RequestId: 5c1e... Version: \$LATEST
pedido: POST /carrinho/total
END RequestId: 5c1e...
REPORT Duration: 2.10 ms Billed Duration: 3 ms Memory Size: 128 MB

simulação simplificada do console

- 1 Na função, aba Monitor › View CloudWatch logs.
- 2 Abra o log stream mais recente, o primeiro da lista.
- 3 Ache pedido: POST /carrinho/total (o seu print) e, na linha REPORT, anote Duration e Billed Duration.

Quanto custa a função

ITEM	CONTA	CUSTO POR MÊS
Chamadas	10.000 visitas × 2 chamadas = 20.000 (o grátis é 1 milhão)	US\$ 0,00
Tempo	20.000 × 0,003 s × 0,125 GB = 7,5 GB-s (o grátis é 400.000 GB-s)	US\$ 0,00
Logs	poucos KB de texto por mês	centavos

FORA DO GRÁTIS

US\$ 0,20 por milhão de chamadas e US\$ 0,0000166667 por GB-s, os valores verificados na Aula 7.

Checkpoint: logs lidos

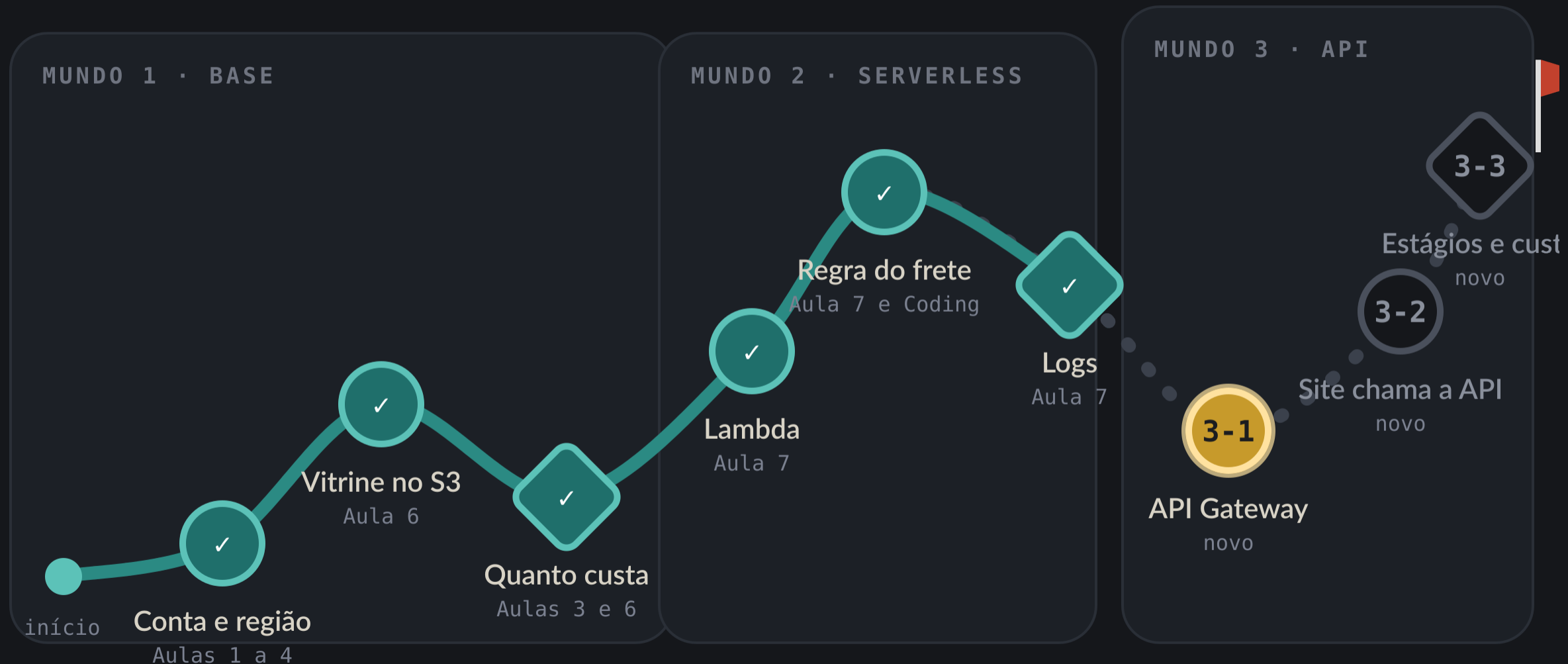
- ◆ A linha do `print` encontrada no CloudWatch.
- ◆ Duration e Billed Duration anotados no relatório de custo.

CONFIRA

Você encontrou `pedido: POST /carrinho/total` no log.

TRAVOU NESTA FASE?

Rode de novo o teste `carrinho` no console e recarregue os logs: o log stream novo aparece no topo.



FASE 3-1 · REVISITA CONTEÚDO NOVO

Uma API na frente da função

O chefe do segundo mundo: dar à loja um endereço público com rotas, como toda API de verdade.

AO FINAL DA FASE

GET /produtos respondendo pelo endereço da API

FASE 3-1 · PARA COMEÇAR

Na Aula 7 a função já tinha uma URL. Para que um API Gateway?

Uma URL por função vira bagunça quando a loja tem dez funções. A API dá um endereço só, com rotas organizadas, controle de acesso, limite de chamadas, CORS e versões (estágios).

RELEMBRE

CONTEÚDO NOVO

API e rota

API é a porta que um programa oferece para outros programas conversarem com ele. **Rota** é a combinação de **método HTTP + caminho**, e cada rota leva a uma ação.

PENSE ASSIM

A recepção de uma loja: um endereço só, e a recepcionista encaminha cada pedido para o setor certo.

exemplo

```
GET /produtos      -> lista a vitrine  
POST /carrinho/total -> calcula o total
```

```
https://abc123.execute-api.us-east-1.amazonaws.com/produtos  
|----- endereço da API -----| | rota |
```

RELEMBRE

CONTEÚDO NOVO

Verbos e códigos HTTP

O **verbo** diz a intenção do pedido; o **código** da resposta diz o que aconteceu.

CUIDADO

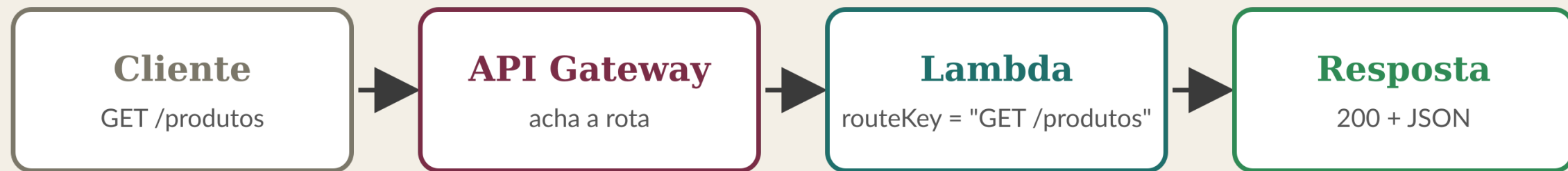
Os códigos 4xx são erro de quem pediu; os 5xx, erro de quem respondeu.



exemplo

GET	buscar dados	200	deu certo
POST	enviar dados	404	rota não existe
PUT	atualizar	403	sem permissão
DELETE	apagar	500	erro dentro do servidor

O caminho de um pedido



- ◆ O API Gateway transforma o pedido HTTP num **evento** e chama a Lambda.
- ◆ Por isso o evento de teste da Fase 2-1 tinha `routeKey`: é exatamente o que chega agora.
- ◆ Usamos o tipo **HTTP API**: mais simples e mais barato que o REST API, cerca de US\$ 1,00 por milhão de chamadas.

Criar a HTTP API: tipo e integração

Console AWS Services Search N. Virginia aluno-dev

API Gateway > APIs > Create API

Create API

1 HTTP API
Simples e barata. Botão Build.

WebSocket API
Conexão aberta, tempo real.

REST API
Mais recursos, mais cara.

Step 1 · Create and configure integrations

2 Integrations
Lambda

3 Lambda function
loja-api

4 API name
loja-api

AWS Region
us-east-1

Cancel

5

Next

simulação simplificada do console

- 1 **Create API** › em **HTTP API**, clique em **Build**.
- 2 **Add integration** › **Lambda**.
- 3 Região us-east-1, função `loja-api`.
- 4 Nome da API: `loja-api`.
- 5 **Next**.

Rotas e estágio

Console AWS

Services

Search

N. Virginia

aluno-dev

API Gateway > APIs > Create API

Step 2 · Configure routes

1

Method	Resource path	Integration target
GET	/produtos	loja-api
POST	/carrinho/total	loja-api

Add route

Step 3 · Define stages

2

Stage name	Auto-deploy
\$default	ligado

Previous

3

Next

4

Create

simulação simplificada do console

- 1 Duas rotas, as duas com destino `loja-api`: `GET /produtos` e `POST /carrinho/total`.
- 2 Estágio `$default` com **Auto-deploy** ligado.
- 3 **Next**: a tela 4 é só uma revisão.
- 4 **Create**.

Criando pelo console, o API Gateway ganha sozinho a permissão para chamar a função. Pelo terminal, seria preciso `aws lambda add-permission`.

Copiar o endereço da API

Console AWS

Services

Search

N. Virginia ▾

aluno-dev ▾

API Gateway > APIs > loja-api

loja-api

API details

API ID	Protocol	Created
abc123	HTTP	2026-10-08

Stages for loja-api

Stage name	Auto-deploy
\$default	Enabled

Invoke URL

1 <https://abc123.execute-api.us-east-1.amazonaws.com>

simulação simplificada do console

- 1 Copie o **Invoke URL**. É o endereço da loja daqui em diante.
- 2 Teste
no navegador: cole o endereço com `/produtos` no fim.

Sem barra no fim do endereço quando for colar no HTML ou no curl.

Testar a API de fora

terminal

```
$ API=https://SEU-ID.execute-api.us-east-1.amazonaws.com
$ curl $API/produtos
[{"id": 1, "nome": "Camiseta básica", "preco": 39.9, "tamanho": "M"}, ...]
$ curl -X POST $API/carrinho/total -H "Content-Type: application/json" -d '[{"id": 1, "quantidade": 3}, {"id": 2, "quantidade": 1}]'
{"subtotal": 249.6, "frete": 0.0, "total": 249.6}
$ curl $API/qualquer-coisa
{"message": "Not Found"}
```

- ◆ Use o **CloudShell** (ícone >_ no console): no Windows, as aspas do curl mudam. Sem barra no fim do endereço da API.
- ◆ A última chamada mostra que rota não cadastrada nem chega à Lambda: o próprio API Gateway responde 404.
- ◆ No navegador, abrir `$API/produtos` também funciona: é um GET.

FASE 3-1 • CONCLUÍDA

Checkpoint: API no ar

no navegador

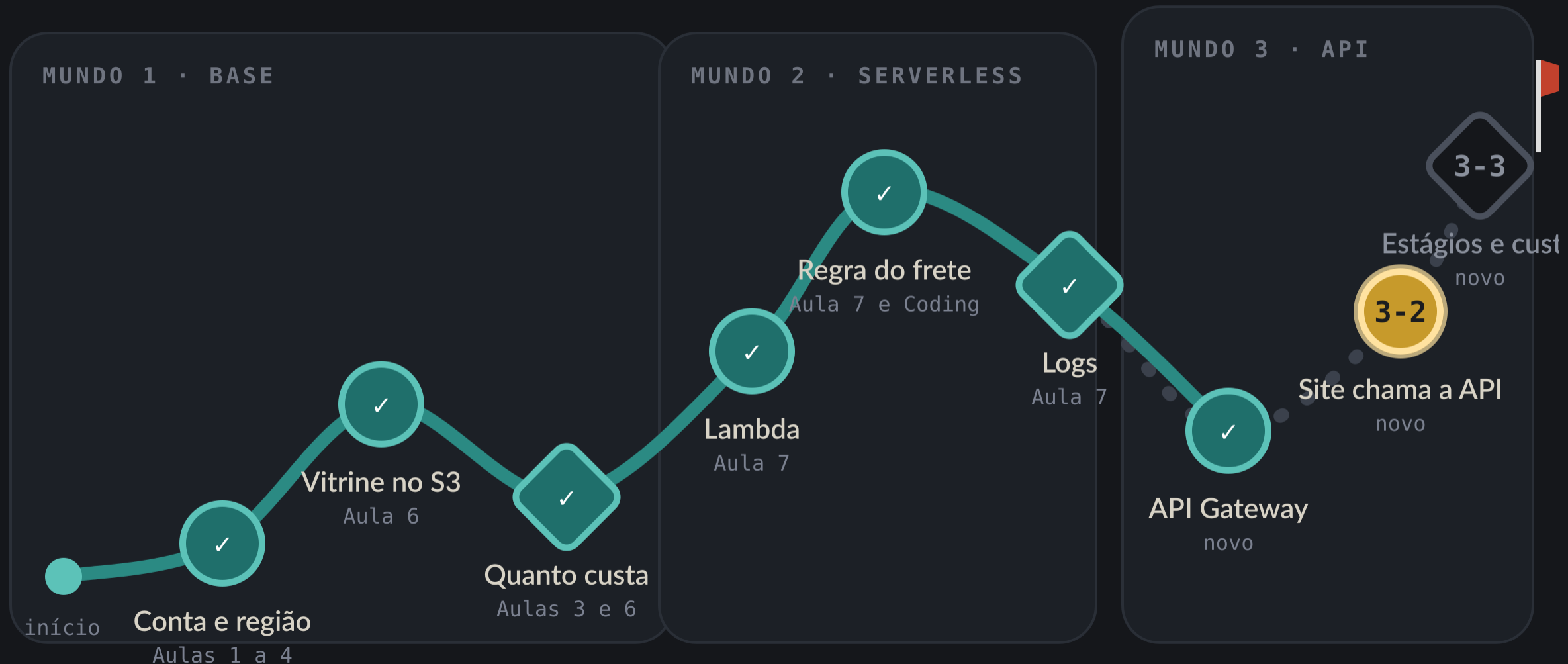
```
https://SEU-ID.execute-api.us-east-1.amazonaws.com/produtos
```

CONFIRA

O endereço da API com `/produtos` mostra o JSON das três peças no navegador.

TRAVOU NESTA FASE?

Confira se as duas rotas apontam para `loja-api` em **Routes**, e se o código da função é o da Fase 2-2 (com `routeKey`).



FASE 3-2 · REVISITA CONTEÚDO NOVO

O site chama a API

A página no S3 deixa de ter preços fixos e passa a perguntar tudo para a API.

AO FINAL DA FASE

a vitrine no S3 mostrando os produtos e o total vindos da API

RELEMBRE

CONTEÚDO NOVO

CORS

Por segurança, o navegador só deixa uma página ler a resposta de **outro endereço** se esse endereço autorizar. A autorização vem em cabeçalhos da resposta: é o CORS.

CUIDADO

Com CORS errado, o curl funciona e o navegador não. O erro aparece no console do navegador (F12), não na AWS.

exemplo

página: `http://loja-roupas-ana-2026.s3-website-us-east-1.amazonaws.com`

API: `https://abc123.execute-api.us-east-1.amazonaws.com`

endereços diferentes -> o navegador pergunta à API:

"posso?" -> `Access-Control-Allow-Origin: *`

Ligar o CORS

Console AWS

Services

Search

N. Virginia ▼

aluno-dev ▼

API Gateway > APIs > loja-api > CORS

Configure CORS

Cross-Origin Resource Sharing

1 Access-Control-Allow-Origin

*

3 Access-Control-Allow-Methods

GET, POST ▼

Access-Control-Max-Age

0

2 Access-Control-Allow-Headers

content-type

Access-Control-Expose-Headers

Access-Control-Allow-Credentials

NO ▼

Cancel

4

Save

simulação simplificada do console

1 Menu **CORS** > **Configure**. Origin: `*`.

2 Headers: `content-type`.

3 Methods: `GET` e `POST`.

4 **Save**.

Por que content-type? Antes do POST com JSON, o navegador faz uma pergunta prévia (OPTIONS) e precisa ouvir que esse cabeçalho é aceito.

index.html: a vitrine vem da API

index.html (do <body> para baixo; o <head> é o mesmo da Fase 1-2)

```
<body>
  <h1>Loja de Roupas</h1>
  <div id="vitrine">Carregando a vitrine...</div>
  <button id="calcular">Total de 3 camisetas e 1 calça</button>
  <p id="total"></p>

  <script>
    const API = "COLE_AQUI_O_ENDERECO_DA_API"; // sem a barra no final

    fetch(API + "/produtos")
      .then((r) => r.json())
      .then((produtos) => {
        document.getElementById("vitrine").innerHTML = produtos
          .map((p) => `<div class="peca">${p.nome} (${p.tamanho}): R$ ${p.preco.toFixed(2)}</div>`)
          .join("");
      });

    document.getElementById("calcular").onclick = () => {
      fetch(API + "/carrinho/total", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify([
          { id: 1, quantidade: 3 },
          { id: 2, quantidade: 1 }
        ])
      })
        .then((r) => r.json())
        .then((c) => {
          document.getElementById("total").textContent =
            `Subtotal R$ ${c.subtotal} · frete R$ ${c.frete} · total R$ ${c.total}`;
        });
    };
  </script>
</body>
</html>
```

- ◆ Troque `COLE_AQUI_O_ENDERECO_DA_API` pelo seu Invoke URL, sem barra no fim.
- ◆ `fetch` faz o pedido; `.then` espera a resposta e monta as peças na tela.
- ◆ O botão faz o POST com 3 camisetas e 1 calça, o mesmo carrinho dos testes.
- ◆ Suba o arquivo no bucket, substituindo o antigo.

A vitrine fica em "Carregando..." para sempre. Onde olhar?

console do navegador (F12)

```
Access to fetch at 'https://abc123.execute-api...' from origin  
'http://loja-roupas-ana-2026.s3-website...' has been blocked by CORS policy
```

É o CORS: a API não autorizou o endereço do site.

Confira a configuração do slide anterior e recarregue a página com **Cmd + Shift + R** (Ctrl + Shift + R no Windows). Se o erro fosse 404, o problema seria o endereço da API colado no HTML.

Checkpoint: loja completa

no navegador

Loja de Roupas

Camiseta básica (M): R\$ 39.90

Calça jeans (G): R\$ 129.90

Moletom (P): R\$ 159.90

[Total de 3 camisetas e 1 calça]

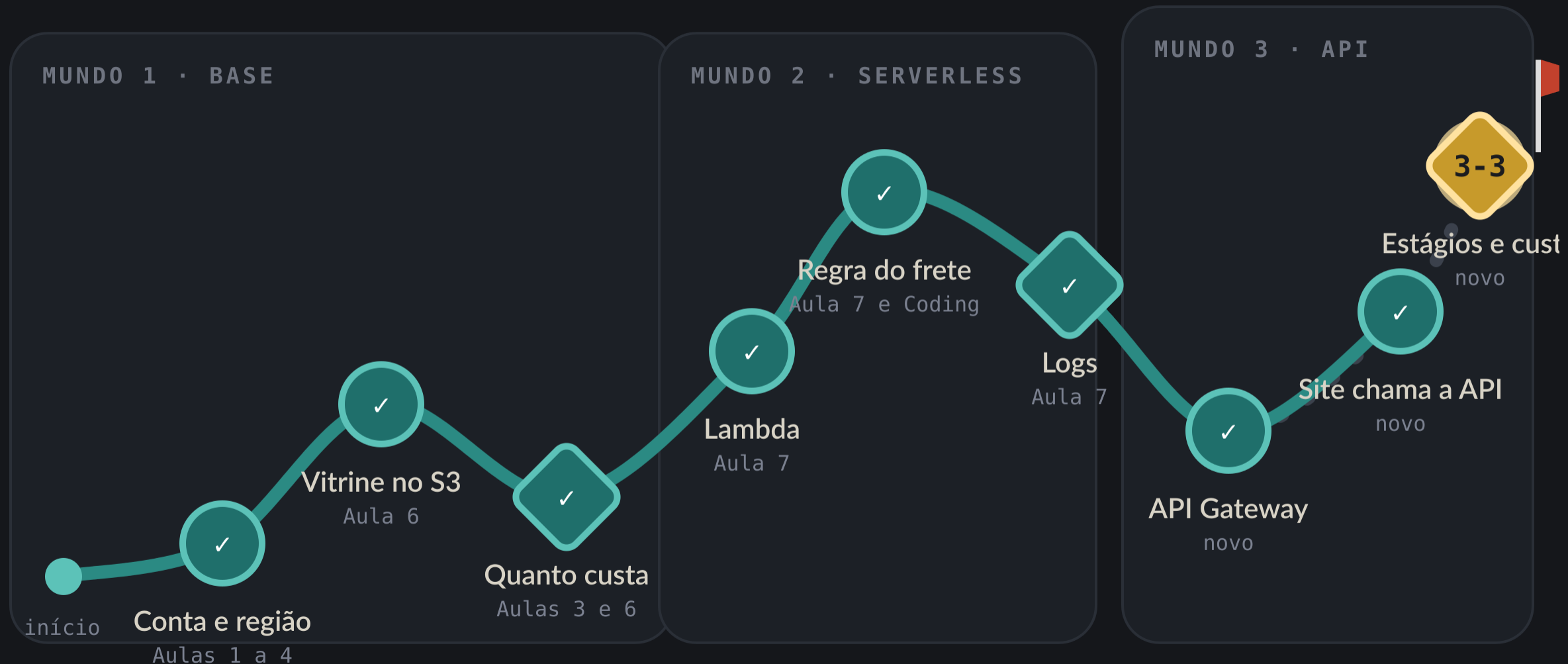
Subtotal R\$ 249.6 · frete R\$ 0 · total R\$ 249.6

CONFIRA

O site do S3 mostra as peças vindas da API e o botão calcula o total.

TRAVOU NESTA FASE?

Baixe `loja-na-nuvem-arquivos.zip` em Materiais › Cloud, no site, e use o arquivo da pasta desta fase. Pasta `fase-3-2`: `index.html` (troque o endereço da API).



FASE 3-3 · REVISITA CONTEÚDO NOVO

Estágios, custo e limpeza

O último chefe: entender versões de uma API e fechar a conta da arquitetura completa.

AO FINAL DA FASE

o custo da loja inteira no relatório e a lista do que apagar depois

RELEMBRE

CONTEÚDO NOVO

Estágio

Uma API pode ter várias versões publicadas ao mesmo tempo, os **estágios**. É comum ter **dev** para testar e **prod** para os clientes. O estágio **\$default** não aparece no endereço.

PENSE ASSIM

O provador da loja (dev) e a vitrine da rua (prod): você testa a roupa num antes de expor no outro.

CUIDADO

Com Auto-deploy no estágio dos clientes,

exemplo

`https://abc123.execute-api.us-east-1.amazonaws.com/produtos ($default)`

`https://abc123.execute-api.us-east-1.amazonaws.com/dev/produtos (dev)`

Auto-deploy ligado: cada mudança na API publica na hora.

A loja inteira, 10 mil visitas por mês

SERVIÇO	USO	CUSTO POR MÊS
S3	10.000 leituras de uma página pequena	menos de US\$ 0,01
API Gateway (HTTP API)	20.000 chamadas × US\$ 1,00 por milhão	US\$ 0,02
Lambda	20.000 chamadas, dentro do grátis	US\$ 0,00
CloudWatch	poucos KB de log	centavos
Total		alguns centavos de dólar

COMPARE COM A AULA 6

Um EC2 pequeno ligado o mês inteiro custa alguns dólares, com 10 visitas ou com 10 mil. Na arquitetura serverless, o custo acompanha o uso.

Limpeza, nesta ordem

- ◆ API Gateway: selecione `loja-api` › **Delete**.
- ◆ Lambda: `loja-api` › **Actions** › **Delete**.
- ◆ CloudWatch › **Log groups**: apague `/aws/lambda/loja-api`.
- ◆ S3: **Empty** no bucket e depois **Delete**.
- ◆ Não apague hoje o que a equipe vai usar no projeto. A limpeza é para o fim do semestre ou para testes que não servem mais.

Checkpoint final

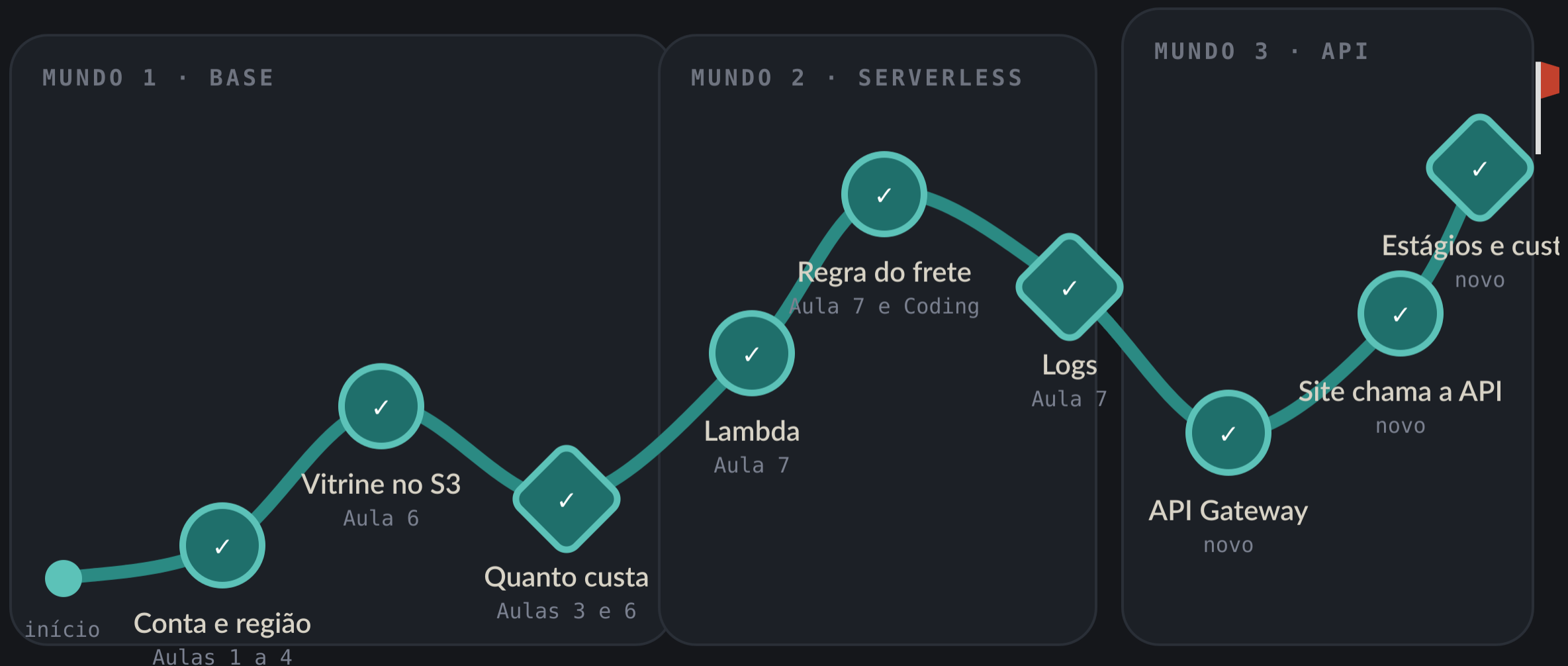
- ◆ Relatório de custo com S3, Lambda e API Gateway.
- ◆ Endereço da API e do site anotados no README da equipe.
- ◆ Lista de limpeza guardada para o fim do semestre.

CONFIRA

A loja funciona de ponta a ponta e o relatório explica quanto ela custa.

TRAVOU NESTA FASE?

Revise as fases 1-2, 2-2 e 3-1: são as que mais travam. Os arquivos de todas estão no zip.



O MAPA DE HOJE

Mapa completo

Da conta conferida a uma loja com vitrine, regras e API, pagando centavos por mês.

O que cada fase revisitou

FASE	CONCEITO	AULA
1-1	nuvem segundo o NIST, camadas, região e zona	Aulas 1, 2 e 4
1-2	bucket, objeto, site estático e política pública	Aula 6
1-3	pagar pelo uso, CAPEX e OPEX	Aulas 3 e 6
2-1	Lambda, evento e JSON	Aula 7
2-2	regra de negócio reaproveitada do Coding	Aula 7
2-3	logs, início a frio e custo da função	Aula 7
3-1	API, rota, verbos e códigos HTTP	novo
3-2	CORS e site consumindo a API	novo
3-3	estágios, custo da arquitetura e limpeza	novo

Checklist para levar ao projeto de vocês

- ◆ Uma rota `GET` que devolve os dados principais do domínio da equipe, em JSON.
- ◆ Uma rota `POST` que aplica uma regra de negócio do DDR e devolve o resultado.
- ◆ O site da equipe no S3 consumindo a API, com CORS configurado.
- ◆ O relatório de custo com S3, Lambda e API Gateway, e a suposição de uso escrita.
- ◆ Endereços do site e da API no README.

REFERÊNCIAS

Para estudar depois

- ◆ AWS. Tutorial: Create a CRUD HTTP API with Lambda and DynamoDB. API Gateway Developer Guide.
- ◆ AWS. Configuring CORS for an HTTP API. API Gateway Developer Guide.
- ◆ AWS. Hosting a static website using Amazon S3. Amazon S3 User Guide.
- ◆ AWS. Amazon API Gateway pricing, AWS Lambda pricing, Amazon S3 pricing. Valores de us-east-1; confira antes de orçar.
- ◆ MELL, P.; GRANCE, T. The NIST Definition of Cloud Computing. NIST SP 800-145, 2011.